

TOLERÂNCIA A FALTAS EM
SISTEMAS DISTRIBUÍDOS DE TEMPO REAL
BASEADOS EM REDES LOCAIS

Daniel Brandão Cavalcanti

CERTI
C.P. 5053 CAMPUS UFSC
88041 FLORIANÓPOLIS SC

RESUMO

Nos últimos anos tem sido crescente o uso de redes locais em diversas aplicações. Entretanto, sua utilização em sistemas distribuídos de tempo real exige o desenvolvimento de recursos específicos de tolerância a faltas, de forma a assegurar o atendimento aos requisitos de confiabilidade e de disponibilidade associados a este tipo de aplicação. Neste trabalho são apresentadas algumas técnicas que visam prover tais recursos, e é apresentado um modelo para a sua implementação.

1 - INTRODUÇÃO

Redes locais tem sido desenvolvidas em diversas configurações. Entretanto, padrões internacionais já estão estabelecidos para algumas topologias e métodos de acesso ao meio de transmissão. O termo rede local será utilizado aqui para designar os tipos de redes padronizadas pelo Projeto 802 do IEEE (IEEE 85).

A disponibilidade de componentes VLSI que permitem o acoplamento de microcomputadores a tais redes tem fornecido condições particularmente atraentes para a implementação de sistemas distribuídos.

Diversas características intrínsecas destas redes, tais como a arquitetura descentralizada, possibilidade de reconfiguração (pela conexão ou desconexão de estações), e a conectividade máxima, permitem o seu uso em aplicações que requerem elevada confiabilidade e disponibilidade.

2 - TERMINOLOGIA

Convém definir com precisão (RAND 78) o significado de alguns termos que serão utilizados aqui:

CONFIABILIDADE _ é uma medida do sucesso com que um sistema adere às especificações de seu comportamento, tais como MTBF (tempo médio entre falhas) e MTTR (tempo médio para reparo).

DISPONIBILIDADE _ é a fração do tempo em que um sistema permanece dentro dos limites de sua especificação:

$$\frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}}$$

FALHA _ é o desvio de um sistema em relação à especificação de seu comportamento.

ERRO _ ou "estado errôneo" de um sistema, é aquele em que o estado interno do sistema é tal que existem circunstâncias (dentro das especificações do uso deste sistema) em que a continuidade do funcionamento, pelos algoritmos normais do sistema, levará a uma falha.

FALTA _ é a causa física ou algorítmica de um erro.

3 - TÉCNICAS

Duas abordagens distintas tem sido usadas para se obter confiabilidade em sistemas computacionais. A primeira é evitar faltas, pelo uso de componentes mais confiáveis (intolerância a faltas). A segunda é tolerância a faltas (isto é, a capacidade de um sistema de sobreviver a faltas em seus elementos), em que a confiabilidade é atingida através do uso de redundância protetora (RAND 78). Duas técnicas principais tem sido utilizadas para se obter tolerância a faltas:

- 1) mascaramento (redundância estática).
- 2) recuperação (redundância dinâmica).

A técnica de mascaramento é utilizada para mascarar, ou esconder, os efeitos das faltas. O mascaramento é adequado para o ambiente de tempo real, mas requer que no mínimo se triplique a máquina, como o TMR (Triple Modular Redundancy) de SIFT (WENS 78), em que 3 cópias de uma unidade operam simultaneamente, com um sistema de votação resolvendo o problema de resultados conflitantes através de decisão por maioria. Devido ao alto custo desta solução, seu uso está restrito a aplicações em que confiabilidade extremamente alta é imprescindível.

A redundância dinâmica, por outro lado, é caracterizada pela tentativa de forçar as faltas a se mostrarem, ao invés de tentar mascará-las. Sistemas dinamicamente redundantes tentam rapidamente detectar e diagnosticar a falta, para então realizar os procedimentos necessários para lidar com a falta.

O processo de recuperação dos efeitos de uma falta envolvem:

- 1) detecção e diagnóstico da falta;
- 2) reparação ou reconfiguração, para remover a origem da falta;
- 3) recuperação, isto é, restauração do estado do sistema a uma situação livre de erros, através de técnicas como o "rollback".

O grau de confiabilidade e desempenho que se pode obter de um sistema que utiliza redundância dinâmica pode ser de certa forma inferior ao que se pode atingir com redundância estática. O sistema pode também operar temporariamente de forma incorreta, antes do diagnóstico da falta. Além disto, o tempo necessário para se realizar os procedimentos que levam à recuperação podem reduzir o desempenho do sistema, e um sistema operando com uma falta não diagnosticada pode ter uma atuação degradada. Entretanto, para sistemas em que a preocupação principal é elevada disponibilidade, e em que pequenas degradações de desempenho são aceitáveis, o custo relativamente mais baixo da redundância dinâmica pode torná-la uma solução atraente.

Auto-diagnóstico é uma questão fundamental em sistemas distribuídos que requerem elevada disponibilidade. Um sistema que é dividido em um determinado número de subsistemas, ou unidades, é definido como auto-diagnosticável se ele é capaz de identificar univocamente as unidades com funcionamento incorreto. À medida que faltas são localizadas, tais unidades podem ser reparadas ou substituídas, ou ainda unidades com faltas podem ser isoladas e o sistema reconfigurado.

4 - REDE LOCAL

Embora existam muitas diferenças de implementação entre redes locais, elas compartilham uma série de características de fundamental importância sob o ponto de vista de tolerância a falhas:

- 1) A arquitetura descentralizada permite que os efeitos de qualquer falta sejam restritos apenas à estação onde ela ocorreu;
- 2) O meio de transmissão compartilhado provê conectividade máxima entre as estações. Desta forma quaisquer duas estações podem estabelecer comunicação direta através da rede. Isto aumenta significativamente a capacidade de diagnóstico na rede;
- 3) A facilidade de reconfiguração, pela conexão ou desconexão de estações à rede. Devido à capacidade de reconfiguração, as redes locais em que existe redundância são particularmente adequadas para aplicações que exigem elevada disponibilidade.

O modelo que está sendo proposto aqui é o de uma rede modular e funcionalmente distribuída de unidades independentes interconectadas, como na Figura 1. Cada estação é um microcomputador com memória própria, tanto principal como secundária (um disco "winchester"), além de recursos de acesso à rede (incluindo o transceptor).

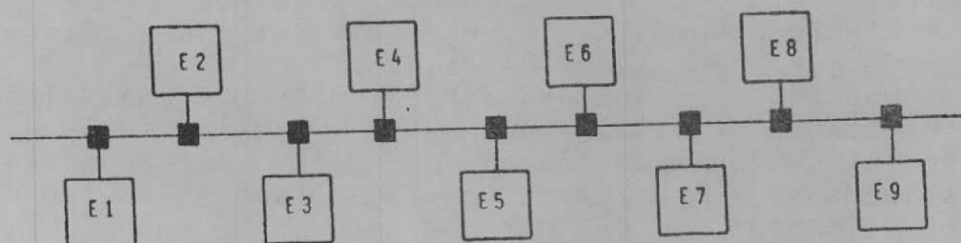


Figura 1 - Rede Local com 9 Estações

O "software" na rede consiste de dois tipos de elementos: módulos do núcleo do sistema operacional que estão replicados em cada estação, e processos que são alocados a estações específicas (isto é, executados pelo processador da estação e com o código armazenado na memória da estação). Os processos comunicam-se por meio de mensagens. As primitivas de comunicação, do tipo SEND e RECEIVE, são executadas por rotinas do núcleo do sistema operacional.

Os protocolos da rede são implementados como processos (processos de rede), e estão estruturados em níveis, de acordo com o modelo de referência ISO/OSI. Em cada estação os processos de rede mantêm informações sobre a localização dos processos de aplicação numa tabela de alocação processo-estação, tornando transparente ao processo de aplicação a localização do processo com o qual está se comunicando. Dependendo dos requisitos de tempo dos processos de aplicação, o acesso à rede pode ser feito pelos níveis mais baixos, transferindo à aplicação a responsabilidade quanto aos mecanismos de controle normalmente fornecidos pelos níveis mais altos dos protocolos de acesso à rede. Uma rede com as características mencionadas aqui está descrita em detalhes em (SAKA 84) e (ARAU 83).

Dados são trocados entre estações através da transmissão de pacotes pela rede. Com relação à confiabilidade, as redes locais possuem características de retransmissão automática pela estação-origem de pacotes não reconhecidos, e descarte automático na estação-destino de pacotes corrompidos durante a transmissão.

5 - FALTAS EM ESTAÇÕES

Para tornar a reconfiguração possível, a redundância deve existir a nível de processos. Processos anteriormente ativos em estações em que ocorram faltas podem assim ser ativados em estações livres de faltas.

Embora a perda de uma estação devido a uma falta não afete seriamente o desempenho de um sistema em que haja redundância, uma única estação com falta operando sem ser diagnosticada pode corromper todo o sistema ao enviar dados errôneos pela rede.

Para se implementar um sistema distribuído auto-diagnosticável, cada estação livre de faltas deve ser capaz de atingir independentemente um diagnóstico correto de todas as faltas em estações na rede. Quando uma estação diagnostica uma falta em outra estação, pode então discriminar a estação com falta, evitando interações com aquela estação. À medida que todas as estações livres de faltas ficam cientes da existência da estação com falta e se recusam a interagir com aquela estação, ela será logicamente isolada das demais estações da rede.

As estações podem tolerar e se recuperar de faltas internas ao empregar técnicas como códigos de correção de erros (ECC) em memória, temporizadores de "watchdog", e verificações de direitos de acesso e violações de limite de segmento (realizados a cada acesso a memória).

Algumas faltas em estações não são identificadas como faltas a nível de sistema. Faltas transitórias de "hardware" podem ser tratadas por repetição. Faltas que perduram após repetições insistentes são consideradas permanentes. Faltas permanentes podem ser mascaradas nas estações onde ocorrem, como por exemplo com ECC em memória, ou isolando blocos de memória a nível de "software". Se as faltas são visíveis fora da estação onde ocorrem, então são tratadas como faltas da estação como um todo.

Considera-se que todas as estações na rede possuem igual probabilidade de falta. A probabilidade de falta no meio físico de transmissão é considerada baixa com relação às das estações.

O diagnóstico de faltas em estações de redes locais pode ser obtido através da implementação de algoritmos distribuídos, baseados em técnicas de difusão ("broadcast") ou outras (CAVA 85), para a disseminação das informações de diagnóstico.

6 - MODELO

Na prática, a implementação de sistemas distribuídos tolerantes a faltas em redes locais se baseia no uso de redundâncias em diversos níveis, além de recursos especiais para a detecção e o mascaramento de faltas, tais como os descritos anteriormente. Esta técnica é denominada de redundância seletiva. A Figura 2 ilustra como a rede local apresentada na Figura 1 poderia estar configurada com recursos de tolerância a faltas.

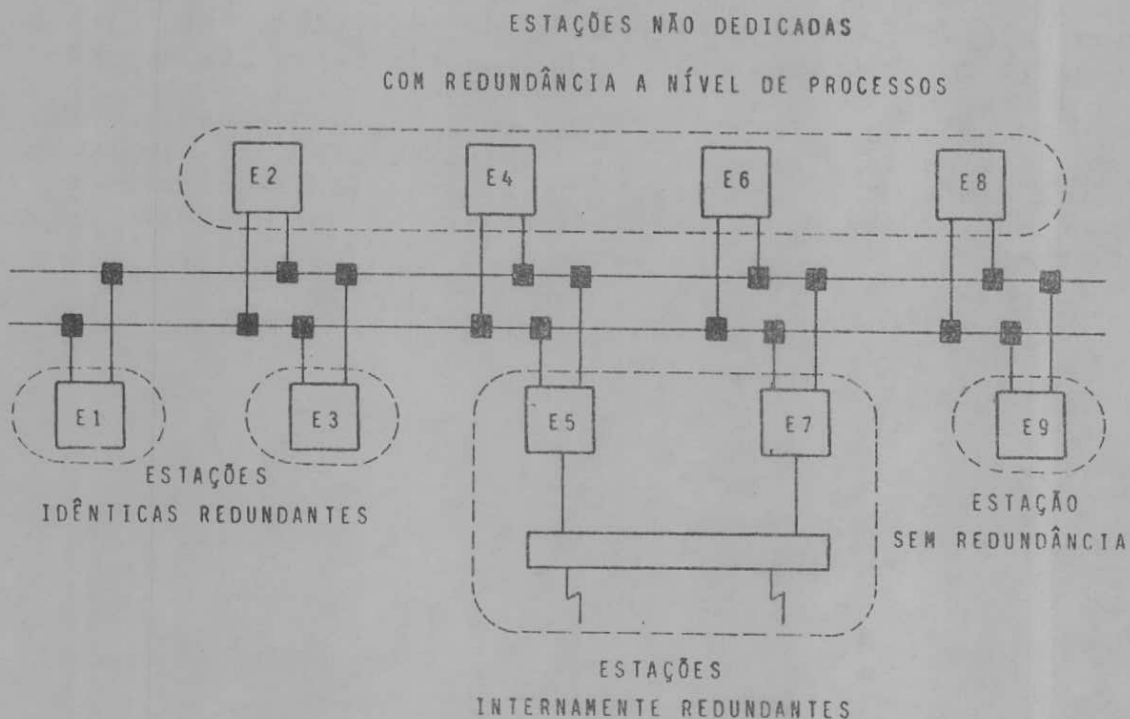


Figura 2 - Rede Local Redundante com 9 Estações.

O uso de redundância seletiva em sistemas distribuídos permite que se concentre recursos de redundância em funções críticas, enquanto funções menos importantes podem utilizar formas mais simples de redundância. Isto se reflete na otimização do sistema tanto em termos da capacidade de processamento como nos custos de implantação.

A rede redundante envolve a duplicação do meio físico de transmissão, e, principalmente, de transceptores e outros recursos de acesso à rede local existentes nas estações (onde as faltas são mais frequentes). Assim, pode-se tolerar faltas individuais em quaisquer destes elementos.

Para funções críticas podem ser utilizadas soluções de estações idênticas redundantes (como no caso de estações servidoras), ou estações internamente redundantes (como no caso de estações de comunicação externa à rede) como forma de incorporar recursos de tolerância a faltas no sistema distribuído.

Para funções menos críticas pode-se adotar soluções de redundância a nível de processos, em que ocorre a replicação dos processos entre estações para lidar com a ocorrência de faltas.

Para funções não essenciais, é possível que não seja adotada redundância alguma. Neste caso, uma estação dedicada incorporaria apenas o algoritmo distribuído de diagnóstico de faltas em estações, recursos internos para detecção de erros, e recursos de acesso à rede redundante.

7 - RECUPERAÇÃO DO SISTEMA

No contexto do sistema distribuído, os mecanismos descritos anteriormente para se chegar à recuperação após a ocorrência de faltas podem ser assim resumidos:

- 1) DETECÇÃO DE ERROS com recursos internos a cada estação, e DIAGNÓSTICO DE FALTAS através de um algoritmo distribuído implementado nas estações da rede;
- 2) RECONFIGURAÇÃO, dependendo do resultado do diagnóstico, envolvendo:
 - 2.1) Desconexão lógica da estação com falta;
 - 2.2) Ativação de réplicas dos processos, antes ativos na estação com falta em estações livres de faltas;
- 3) RECUPERAÇÃO, trazendo todos os processos a um estado consistente nas estações livres de faltas, através de mecanismos de controle de integridade.

É interessante observar que os mecanismos necessários para se implementar o controle de concorrência e o controle de integridade em sistemas distribuídos são de fato complementares. A técnica de ações elementares (atômicas) visa atingir estes objetivos simultaneamente.

8 - TRANSAÇÕES E AÇÕES ATÔMICAS

Ações atômicas são uma generalização do conceito de transações, associadas à implementação de bancos de dados. Uma transação é, por definição, uma sequência de ações que leva um sistema de um estado consistente a outro estado consistente. Para que uma transação se torne atômica ela deve se apresentar como indivisível com relação a faltas e a concorrência.

Isto significa que, ou todos os seus passos são executados e seus efeitos tornados permanentes, e neste caso a transação é "confirmada", ou seus efeitos são desfeitos e a transação é "cancelada".

Além disto, o escalonamento dos diversos passos de transações concorrentes deve ser tal que sua execução tenha o mesmo efeito que uma execução sequencial (escalonamento sequencial).

Uma técnica usual para controle de concorrência é a de bloqueios. Uma transação bloqueia um objeto para torná-lo inacessível a outras transações enquanto o sistema está temporariamente inconsistente. Um mecanismo simples de bloqueio utiliza bloqueios de leitura e escrita. Uma transação usa um bloqueio de leitura se ela somente lê um objeto, e usa um bloqueio de escrita se atualiza um objeto. Bloqueios de leitura podem ser compartilhados, de forma que mais de uma transação pode bloquear o mesmo objeto simultaneamente. Bloqueios de escrita devem ser exclusivos.

Para garantir escalonamento sequencial, todas as transações devem ser "bem formadas" e de "duas fases". Uma transação é bem formada se: bloqueia um objeto antes de acessá-lo, não estabelece bloqueios incompatíveis com outros já existentes, e antes de se completar desbloqueia os objetos que bloqueou. Para ser de duas fases, uma transação nunca deve estabelecer novos bloqueios depois que realiza o primeiro desbloqueio.

Com relação a faltas, uma transação deve sempre se iniciar estabelecendo uma linha de recuperação. Em seguida, realiza uma série de passos e termina com um evento que marca o fim da linha de recuperação. Quando o último passo é executado, o sistema confirma a transação, tornando o efeito de todos os passos permanente, liberando os bloqueios mantidos pela transação, e tornando seus efeitos públicos. Em qualquer momento antes da confirmação, a transação pode ser cancelada. Se a transação é cancelada, todos os efeitos dos passos são desfeitos e o sistema é restaurado ao estado original. Esta técnica é denominada de "rollback".

9 - AÇÕES ATÔMICAS EM SISTEMAS DISTRIBUÍDOS

A implementação de ações atômicas em sistemas distribuídos envolve:

- 1) um mecanismo de controle de concorrência que evite que as ações que estejam fora da ação atômica acessem objetos utilizados pela ação atômica;
- 2) mecanismos de escalonamento que resolvam situações de conflito no acesso a objetos, e que garantam a consistência de execução (escalonamento sequencial);
- 3) mecanismos de atualização de objetos, que no caso de uma ação cancelada, ou de falha durante a confirmação, restaurem o sistema a um estado anterior consistente ("rollback");
- 4) um mecanismo que assegure que, no caso de uma ação atômica confirmada, seu resultado sobreviva a falhas futuras no sistema.

Diferentes mecanismos de controle de concorrência e de escalonamento tem sido desenvolvidos (KOHL 81, CASA 85), podendo ser agrupados nas seguintes categorias:

- a) bloqueio em duas fases;
- b) pré-ordenação, com o uso de senhas ("timestamps");
- c) combinação de bloqueio em duas fases e pré-ordenação.

Os mecanismos necessários para tornar uma ação atômica recuperável devem ser tais que:

- 1) objetos atualizados não sejam liberados até que a ação seja completada;
- 2) os estados iniciais de todos os objetos modificados pela ação possam ser restaurados, o que normalmente envolve a manutenção de uma ata em memória secundária, onde são registradas todas as mudanças em objetos.

A implementação de ações atômicas no contexto de um sistema distribuído exige, além dos mecanismos de controle interno de cada estação da rede, mecanismos de coordenação entre as diversas estações da rede. Estes mecanismos de coordenação devem ser capazes de assegurar que uma ação atômica só seja confirmada, e as modificações nos objetos envolvidos completadas, se todas as estações da rede envolvidas na ação forem capazes de realizar as suas respectivas partes com sucesso.

Uma técnica bastante difundida para implementar estes mecanismos de coordenação é o "protocolo de confirmação em duas fases". Na primeira fase do protocolo é verificado se todos os participantes serão capazes de confirmar suas partes na ação atômica, e na segunda fase informa-se a todos os participantes o estado final da ação atômica (confirmada ou cancelada).

Para aplicação em sistemas distribuídos, uma versão bastante interessante é a denominada "protocolo distribuído de confirmação em duas fases" (MOLI 87). Esta implementação do protocolo de duas fases se dá na seguinte forma:

- 1) a estação que inicia a ação atômica se comunica com todas as estações envolvidas. À medida que cada uma destas estações conclui sua parte, informa o resultado (sucesso ou insucesso) a todas as demais estações envolvidas na ação atômica;
- 2) cada estação aguarda os resultados de todas as demais estações participantes da ação atômica, e então, com base nestes resultados, confirma ou cancela a ação.

Desta forma evita-se a sobrecarga normalmente imposta à estação que inicia a ação atômica, como ocorre no protocolo centralizado de confirmação em duas fases.

10 - REPLICAÇÃO DA BASE DE DADOS

No sistema distribuído discutido aqui algumas estações estarão dedicadas à função de oferecer serviços às demais estações da rede. Um destes serviços é o de base de dados. Como forma de aumentar a disponibilidade do sistema é conveniente que esta base de dados seja replicada, porém não fragmentada. Isto, obviamente, implica em mecanismos específicos de controle de concorrência e integridade a serem implementados.

Os mecanismos descritos anteriormente fornecem a base necessária para outros mecanismos que irão lidar com a questão da replicação. Duas possíveis técnicas para assegurar a coordenação entre as réplicas (cópias) utilizando bloqueios são:

- 1) cópia primária.
- 2) cópias disponíveis.

No algoritmo de cópias disponíveis (MOLI 87), os bloqueios de leitura da base de dados podem ser obtidos em qualquer estação ativa que contenha uma das cópias. Bloqueios de escrita precisam ser obtidos em todas as estações ativas que contenham cópias da base de dados. Cada estação da rede possui numa tabela (T) uma relação do conjunto de estações em que existem cópias da base de dados. Quando uma estação deseja ser acrescentada à tabela (isto é, possui uma cópia) deve executar uma ação atômica para atualizar as tabelas (T) das demais estações da rede, utilizando um protocolo de confirmação de forma a assegurar que o conteúdo da tabela (T) seja idêntico em todas as estações da rede. Quando ocorre uma falta em uma dada estação (F) que contém uma cópia da base de dados, então uma das estações ativas deve executar uma ação atômica para retirar a estação (F) da tabela (T). Quando a estação (F) se recupera da falta, deve requisitar as atualizações necessárias de uma estação ativa onde exista uma cópia da base de dados.

11 - OPERAÇÕES REMOTAS

Existem outros serviços que devem ser fornecidos à rede, e para os quais se utiliza redundância a nível de estação, ou de processos, como forma de aumentar a disponibilidade, mas em que não se necessita de coordenação entre as diversas réplicas (ex: servidor de impressão, servidor de comunicação remota).

Nestes casos, basta que a estação requisitante consiga sucesso na interação com uma, e somente uma, das réplicas. O enfoque aqui passa a ser o de implementar mecanismos eficientes que permitam realizar atomicamente as operações remotas através da rede.

Uma solução interessante é a proposta em (SPEC 82), onde é apresentado um modelo de referencia /operação remota. Neste modelo o processo que realiza a referencia remota é chamado de "mestre" (ou origem) numa estação, e o processo que realiza a operação remota é o "escravo" (ou destino) em outra estação.

Estes processos ativam, nas respectivas estações, os processos de rede de forma a estabelecer uma sessão entre mestre e escravo. Este serviço pode ser implementado no nível de sessão (Nível 5 ISO/OSI) do protocolo de acesso à rede. As diversas ações associadas a uma sessão devem ser realizadas de forma atômica.

Para que uma sessão seja realizada entre mestre e escravo, há inicialmente uma fase de estabelecimento que é em seguida confirmada atômicamente pelo mestre. O mestre faz, então, um pedido de serviço ao escravo, que deve realizá-lo também atômicamente. O escravo retorna a resposta, que será confirmada atômicamente pelo mestre, podendo, em seguida, ser terminada a sessão.

12 - SÍNTESE

Para se obter tolerância a faltas em um sistema distribuído de tempo real baseado em rede local, é preciso integrar os mecanismos de:

- 1) detecção de erros e diagnóstico de faltas
- 2) reconfiguração
- 3) recuperação

O que se propõe aqui é a implementação de:

- a) detecção de erros através de recursos internos a cada estação ("watch-dog", ECC, etc.);
- b) diagnóstico de faltas em estações através de um algoritmo distribuído;
- c) reconfiguração através da ativação de réplicas (existentes pelo uso de redundância seletiva);
- d) recuperação através do uso de ações atômicas, e especificamente:
 - d.1) bloqueios e protocolo de confirmação em duas fases;
 - d.2) algoritmo de cópias disponíveis para acesso à base de dados replicada;
 - d.3) algoritmo de referências/operações remotas no acesso a serviços fornecidos pela rede no contexto do sistema distribuído.

Finalmente, cabe ressaltar que o modelo proposto visa definir um sistema distribuído cuja principal característica é elevada disponibilidade, e está baseado em duas premissas bastante fortes.

Primeiro, assume-se que a probabilidade de falha do meio físico de transmissão é muito baixa. Segundo, assume-se que a probabilidade de faltas simultâneas em múltiplas estações é, também, muito baixa. Portanto, na grande maioria das situações de ocorrência de falta, a origem estará numa estação, e esta será a única estação com falta naquele dado instante. Além disto, esta estação provavelmente já estará reparada e ativa novamente antes que ocorra qualquer outra falta no sistema.

Os recursos de tolerância a faltas fornecerão ao sistema rapidez no diagnóstico de faltas, e evitarão que as suas consequências sejam catastróficas. Em suma, tornarão o sistema mais imune aos efeitos de faltas, que serão eventos raros, porém, inevitáveis.

113 - REFERENCIAS

((ARAU83)

P.P.J. Araújo, D.B. Cavalcanti, e A.S. Cavalcanti, "Supervisão e Controle de Processos: Modularidade em Hardware e Software", Anais do 1. Congresso Latino-Americano de Instrumentação e Controle de Processos, Rio de Janeiro, 1983.

((CASA85)

M.A. Casanova, A.V. Moura, "Princípios de Sistemas de Gerência de Bancos de Dados Distribuídos", Ed. CAMPUS, Rio de Janeiro, 1985.

((CAVA85)

D.B. Cavalcanti, "Fault-Diagnosis in Local Area Networks", CCCNG Report T-140, University of Waterloo, Waterloo, Canada, 1985.

((IEEE85)

IEEE, Std. 802.3 (CSMA/CD) ISO/DIS 8802/3, Std. 802.4 (TOKEN BUS) ISO/DIS 8802/4, Std. 802.5 (TOKEN RING) ISO/DIS 8802/5, IEEE/John Wiley & Sons Inc., New York, 1985.

((KOHL81)

W.H. Kohler, "A Survey of Techniques for Synchronization and Recovery in Decentralized Computer Systems", ACM Computing Surveys, vol. 13 (n. 2), pp. 149-183, 1981.

((MOLI87)

H.G. Molina and R.K. Abbott, "Reliable Distributed Database Management", Proceedings IEEE, vol. 75 (n. 5), 1987.

((RAND78)

B. Randell, P.A. Lee, and P.C. Trealeven, "Reliability Issues in Computing Systems Design" ACM Computing Surveys, vol. 10 (n. 2), pp. 123-165, 1978.

((REED83)

D.P. Reed, "Implementing Atomic Actions on Decentralized Data", ACM Transactions on Computer Systems, vol. 1 (n. 1), 1983.

((SAKA84)

S.R. Sakata, D.B. Cavalcanti, et. al., "Sistema Distribuído de Processamento - SDP86", Anais do Simpósio Brasileiro de Redes de Computadores, Campina Grande, 1984.

(SPEC 82)

A.Z. Spector, "Performing Remote Operations Efficiently on a Local Computer Network", Communications of ACM, vol. 25 (n. 4), 1982.

(SVOB 84)

L. Svobodova, "Resilient Distributed Computing", IEEE Transactions on Software Engineering, vol. SE-10 (n. 3), 1984.

(WENS 78)

J.H. Wensley, "SIFT: Design and Analysis of Fault-Tolerant Computer for Aircraft Control", Proceedings IEEE, vol. 66 (n. 10), pp. 1240-1255, 1978.